

AIGW – 推理服务智能枢纽的现状与未来

蚂蚁集团 / 朱德江

Mooncake 核心成员、Envoy Golang Maintainer

2025/11/15



CONTENT

目录

01 推理场景对网关的挑战

02 AIGW 技术方案选型

03 AIGW 未来发展规划

AIGW — 推理服务智能枢纽



- 降低时延
- 提升吞吐
- 智能路由
- 过载保护
- 多租 QoS
- 自动故障转移

挑战：推理服务 vs 常规服务

➤ 流量特征差异：

- 长连接（单请求可能几十分钟）
- 流式输出 token

➤ 计算负载差异：

- 负载均衡
- 限流

➤ 业务要求：

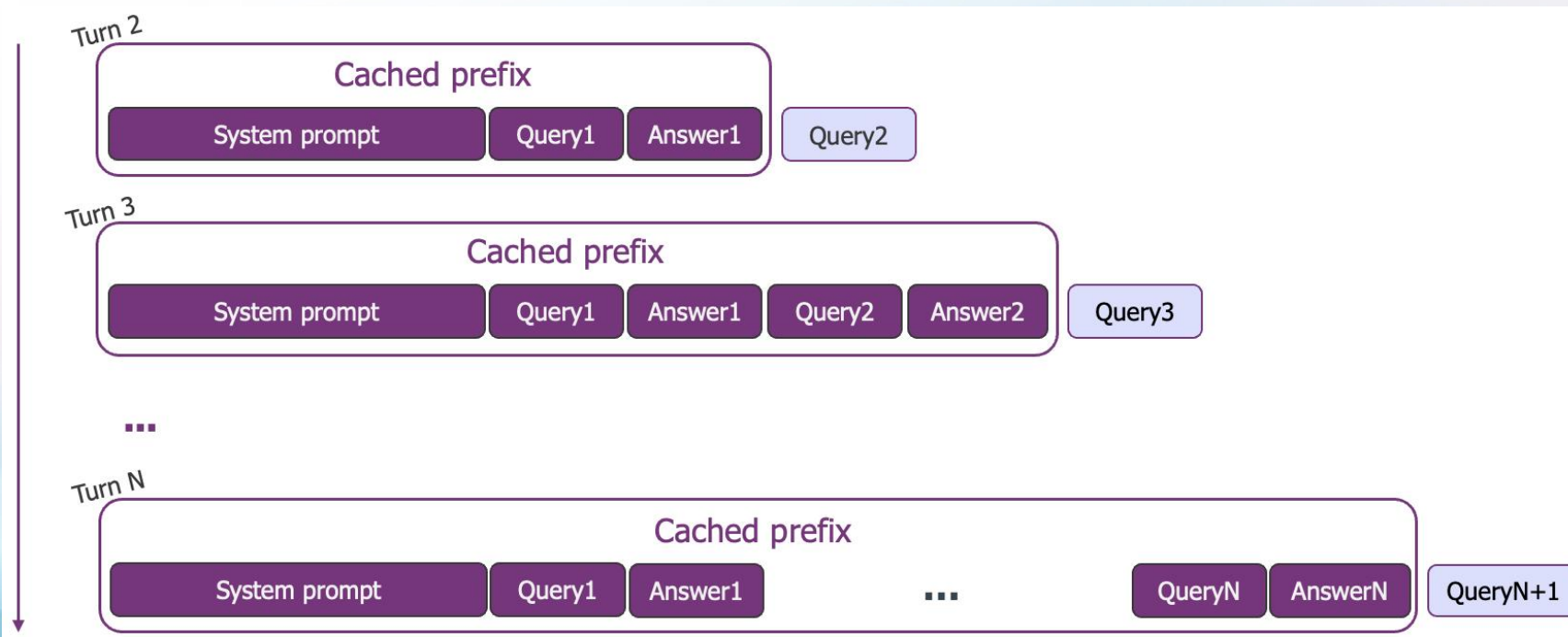
- TTFT
- TPOT

➤ 稳定性要求：

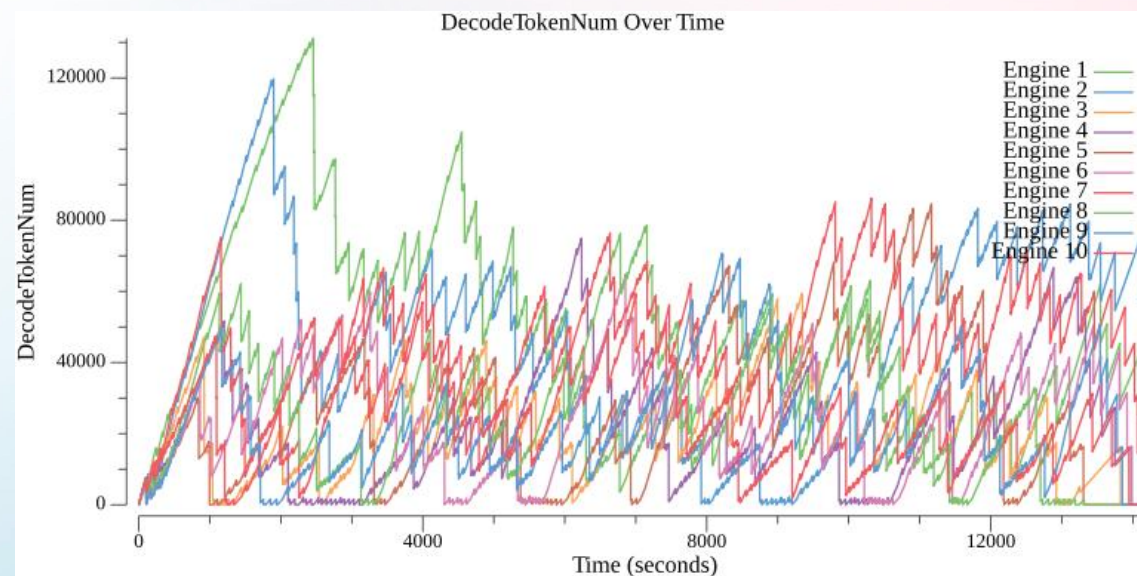
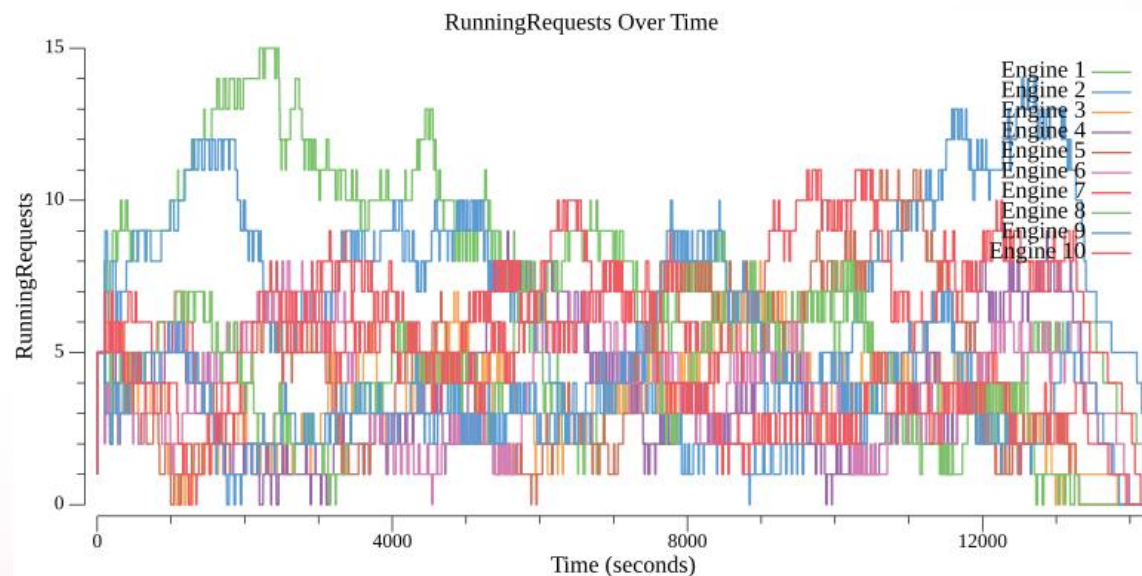
- 时延 SLO
- 每请求排查

挑战：计算负载与请求量是非线性关系

- 计算量大、单点并发小
- 请求 input/output 变化很大、负载波动大
- prefix 语义 cache 复用

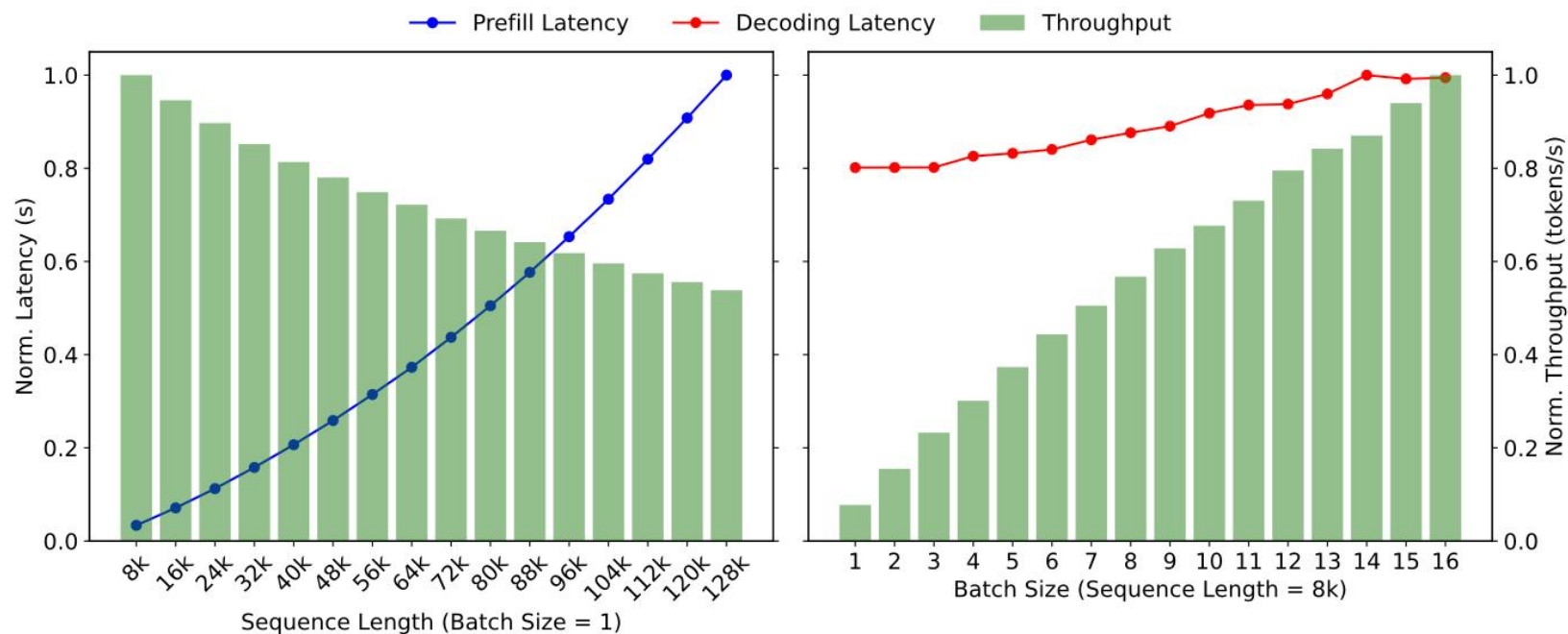


挑战：传统经典算法不再适用



线上流量使用 Round-robin 策略回放，请求数和 Token 用量都很不均衡

计算负载：Prefill vs Decode



Prefill

- 计算 bound
- 无并发能力 (几乎)

Decode

- 组 batch
- 小并发

计算负载： Attention vs FFN

Model	KV/State Memory Access (bytes)	Attention Computation w/o Linear (FLOPs)	Linear before and after Attention (FLOPs)	FFN Computation (FLOPs)
DSv3	2.88×10^8	1.47×10^{11}	2.28×10^{10}	4.84×10^{10}
Kimi K2	2.88×10^8	7.37×10^{10}	1.23×10^{10}	4.84×10^{10}
Qwen3 MoE	7.89×10^8	2.52×10^{10}	1.34×10^{10}	2.84×10^{10}
Step-3	2.56×10^8	3.27×10^{10}	2.07×10^{10}	5.33×10^{10}

Table 2: Theoretical computation and memory access per decoding token at 8K context length.

Attention

- 计算、访存：context length 正相关

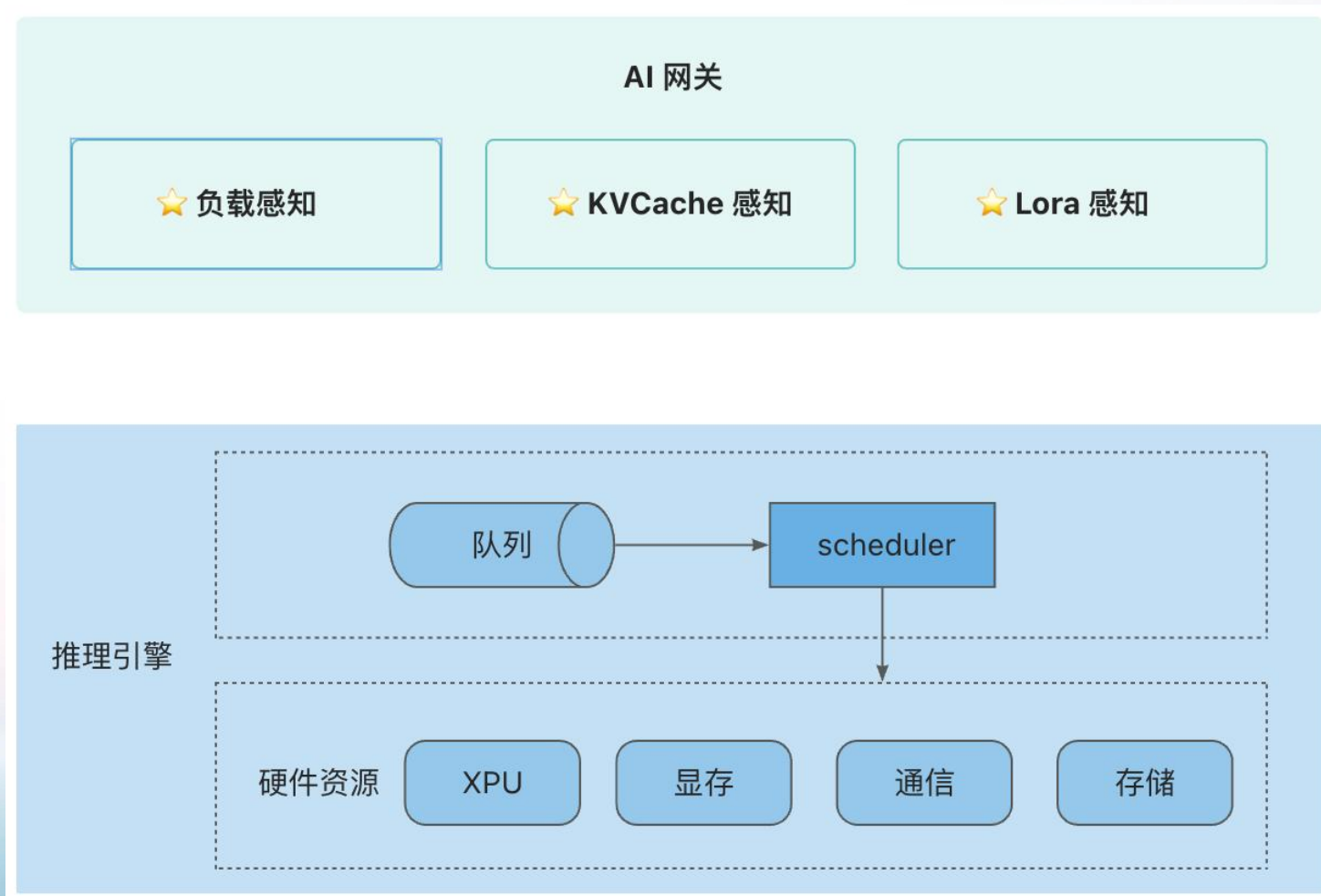
Model	KV/State Memory Access (bytes)	Attention Computation w/o Linear (FLOPs)	Linear before and after Attention (FLOPs)	FFN Computation (FLOPs)
DSv3	1.15×10^9	5.89×10^{11}	2.28×10^{10}	4.84×10^{10}
Kimi K2	1.15×10^9	2.95×10^{11}	1.23×10^{10}	4.84×10^{10}
Qwen3 MoE	3.15×10^9	1.01×10^{11}	1.34×10^{10}	2.84×10^{10}
Step-3	1.02×10^9	1.31×10^{11}	2.07×10^{10}	5.33×10^{10}

Table 3: Theoretical computation and memory access per decoding token at 32K context length.

FFN

- 计算：Batch-size 正相关
- 访存：基本固定

智能路由本质：request -> token 调度协同优化



Request
粒度

- 时延
- 吞吐

Token
粒度

- 组 batch
- 充分利用硬件资源

CONTENT

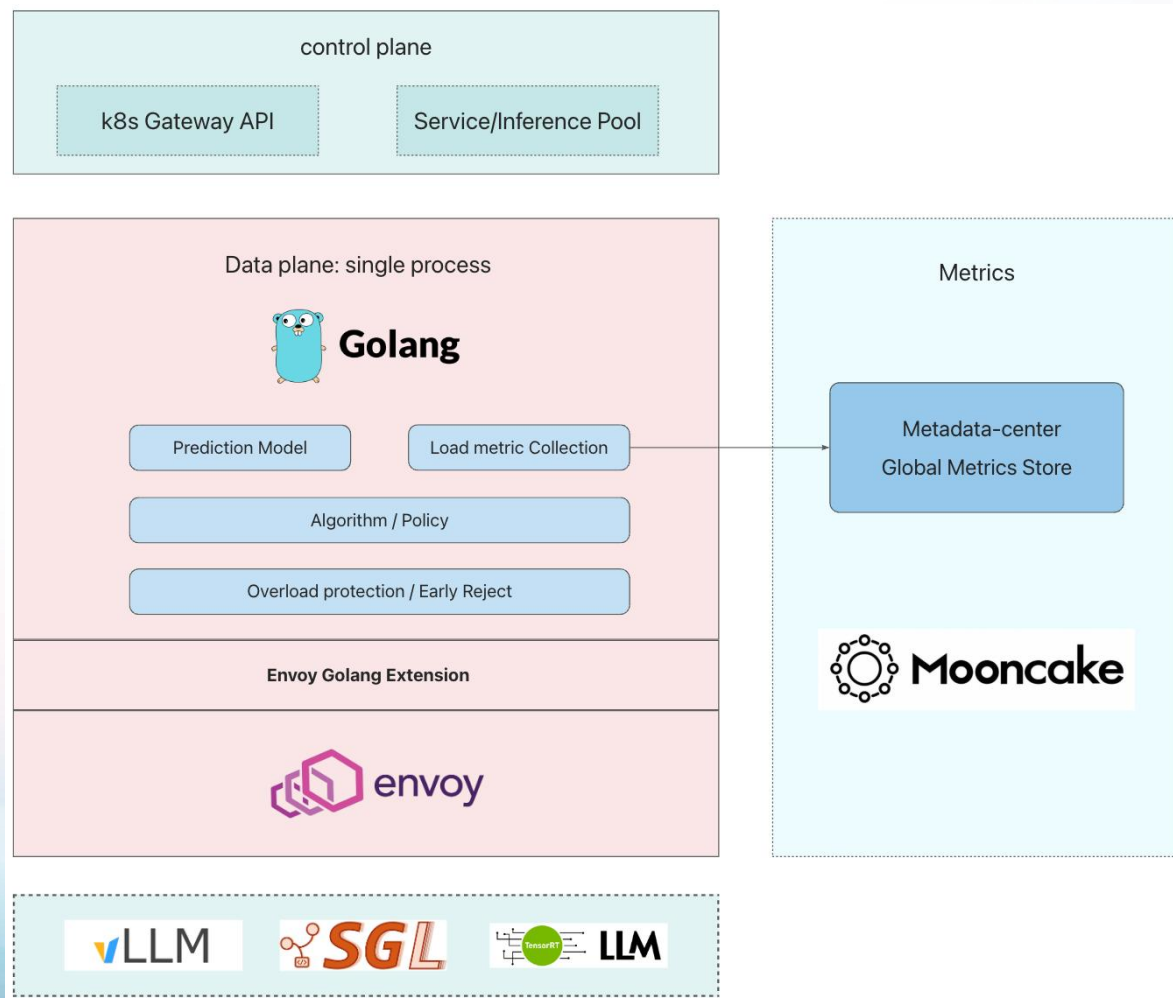
目录

01 推理场景对网关的挑战

02 AIGW 技术方案选型

03 AIGW 未来发展规划

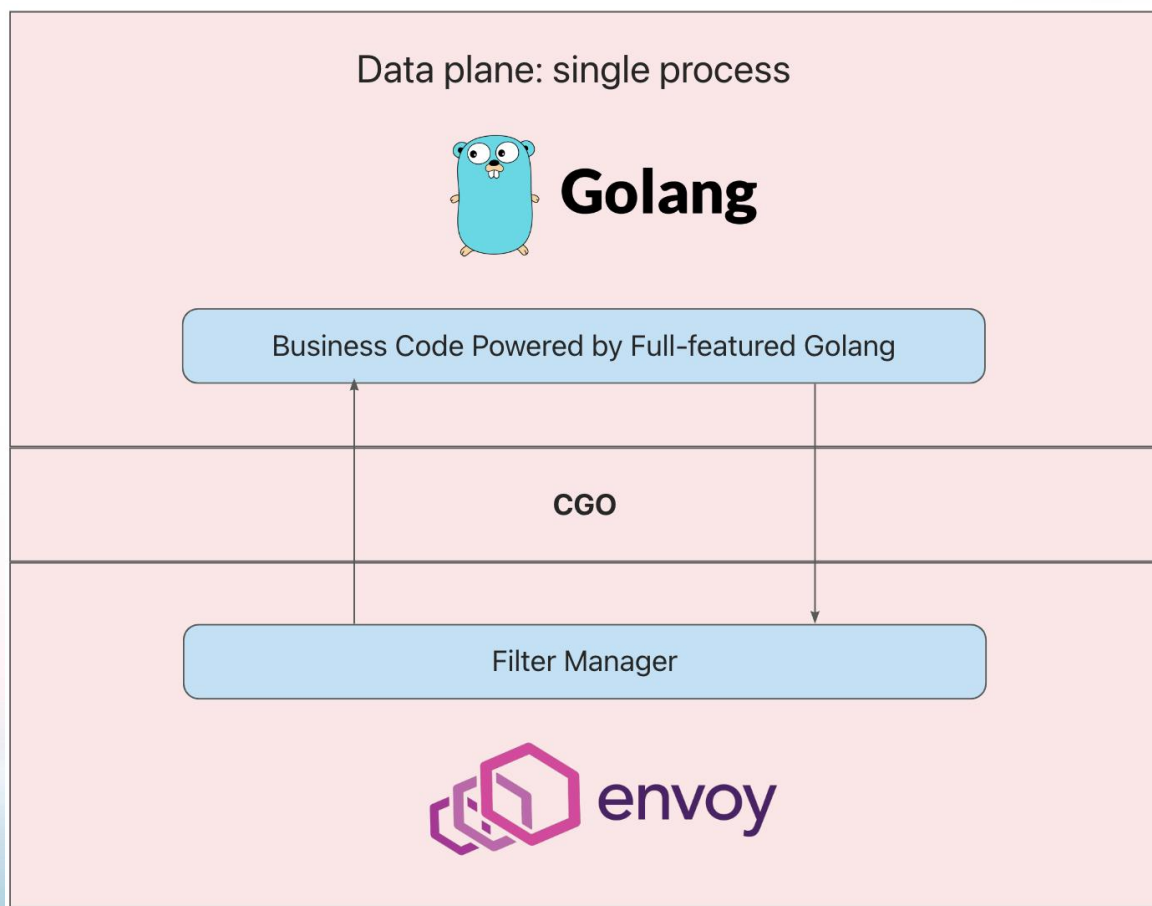
AIGW 技术架构



亮点:

- 灵活 & 强大的 Golang 扩展
- 准实时指标采集机制
- 均衡的负载均衡策略
- 高可用架构

亮点一： Envoy Golang 扩展机制



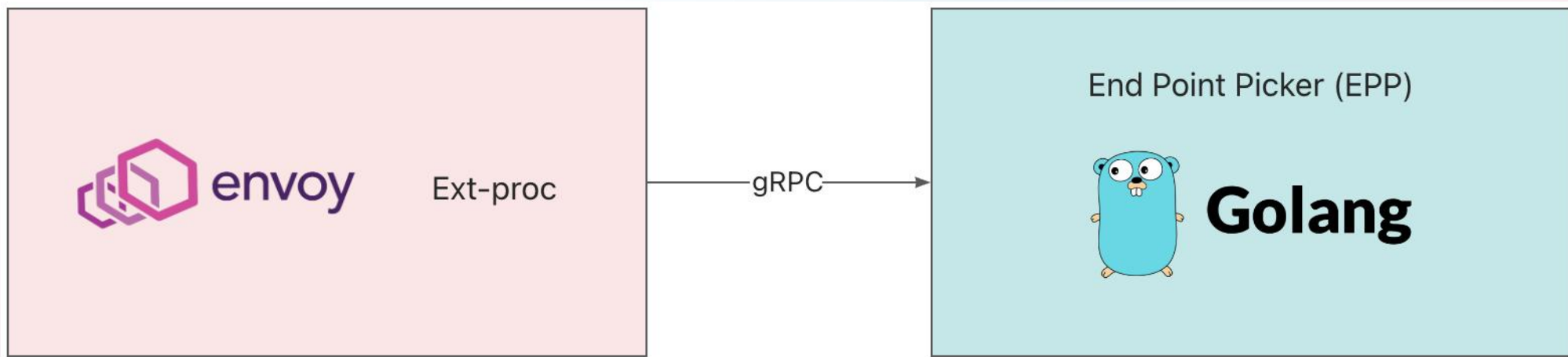
同进程内:

Golang 代码编译为 so 文件，由 Envoy 动态加载，运行在同一个进程内

- 全功能的 Golang 语言支持（不限制 tinygo）
- 完善 & 强大的请求控制：LB 策略 & 请求流程控制
- 易于维护
- 高性能：同进程内的

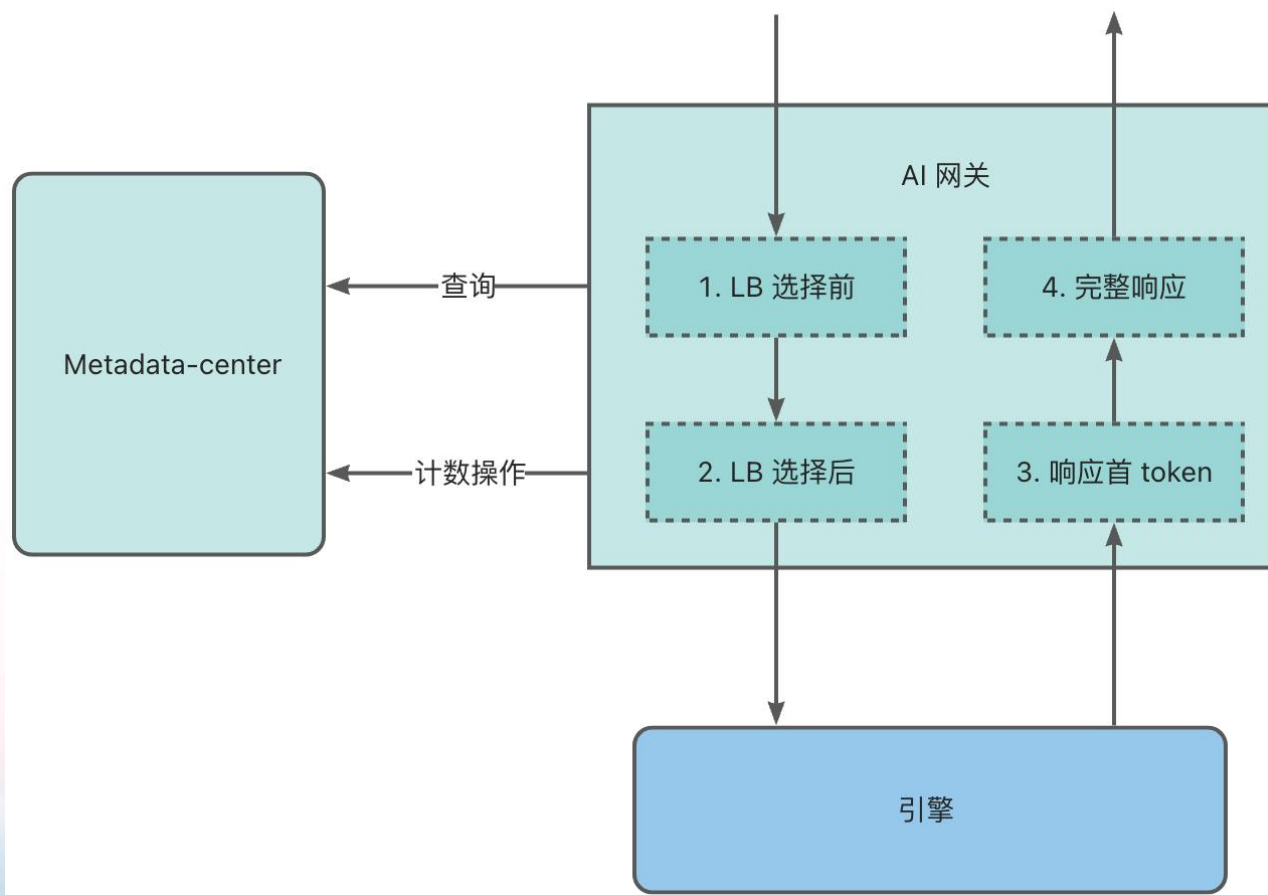
对比：ext-proc 扩展机制

通过 gRPC 与外部进程通讯



- 多一跳网络时延
- 多一个维护组件

亮点二：准实时指标采集机制



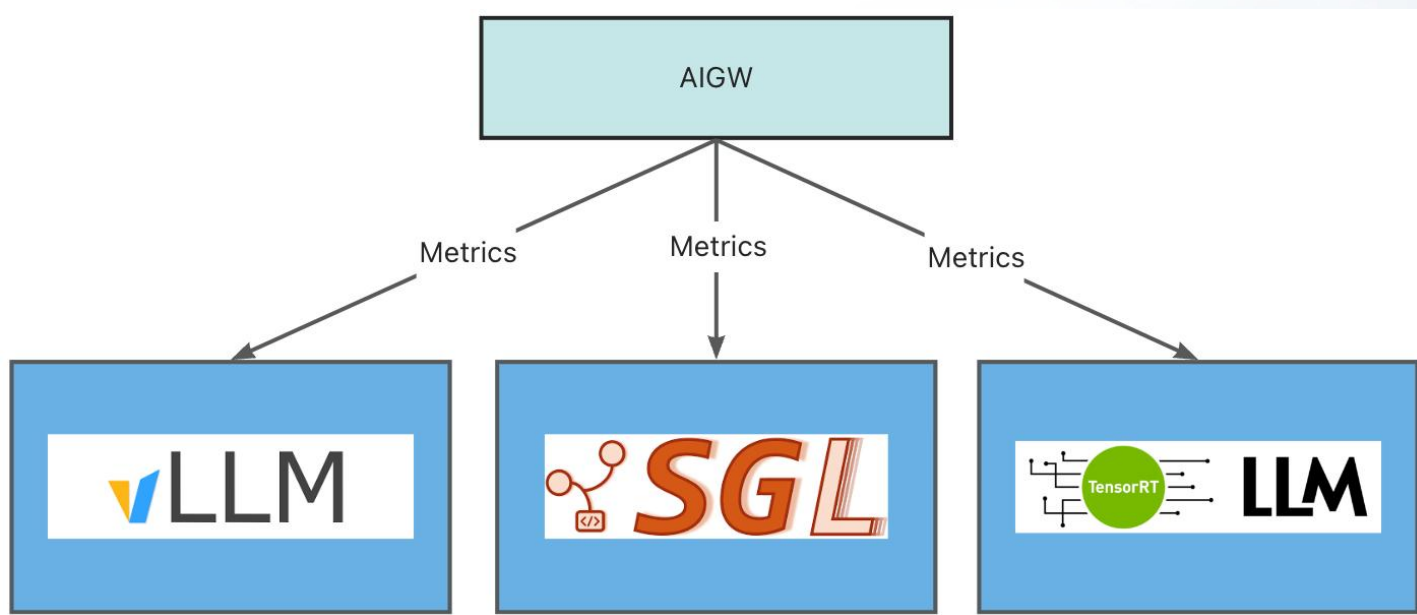
指标 (per Instance) :

- Prefill 请求数
- Prefill Prompt 长度
- 总请求数
- Total Token Number (WIP)

实现机制:

- 计数器
- 协议感知 (prompt)
- 基于灵活的 Golang 扩展机制

对比：周期性从引擎采集 Metrics 指标



劣势:

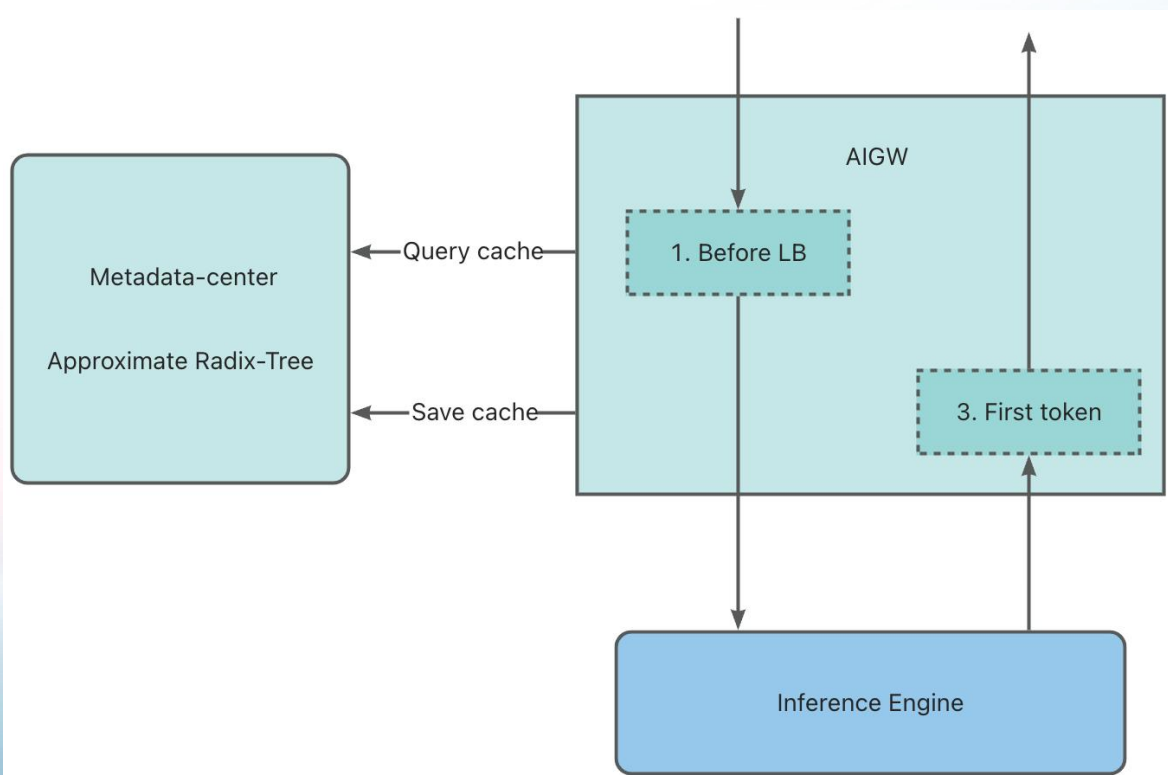
- 时效性差
- 多引擎适配成本
- 周期性采集成本对引擎的开销

时效性差，容易导致出现导致更新周期内，选择同样的节点

近似 cache-aware: 全局近似前缀树

目标: 尽可能命中引擎侧 prefix KVCache 本地存储

引擎: 本地 Radix Tree: 索引本地 KVCache



AIGW:

Prompt 文本切成 chunk, 计算 hash

Metadata-center:

全局索引匹配 => 不同引擎上匹配的长度

近似:

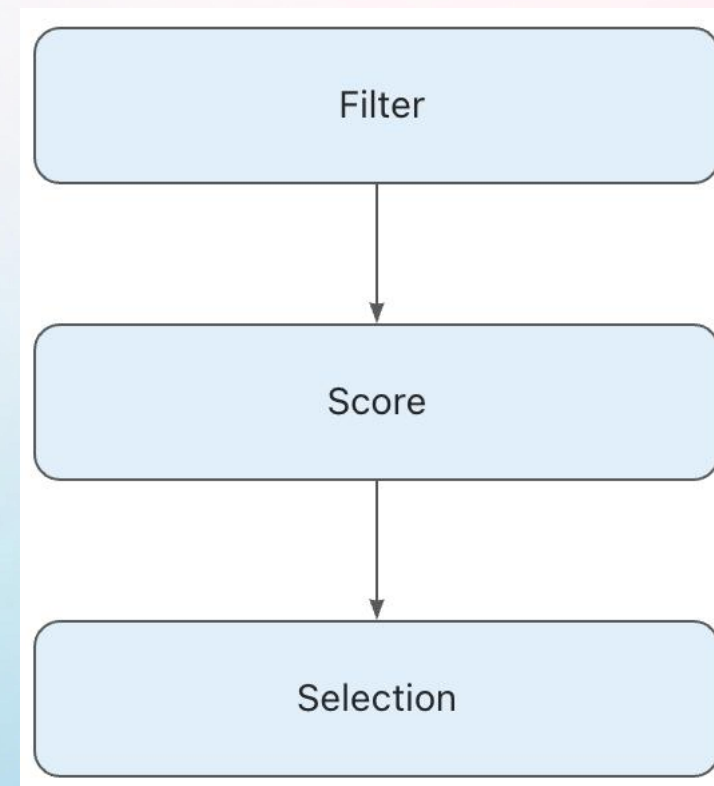
- 文本 => token
- 近似 GC => LRU 淘汰

亮点三：多因子综合权重决策算法

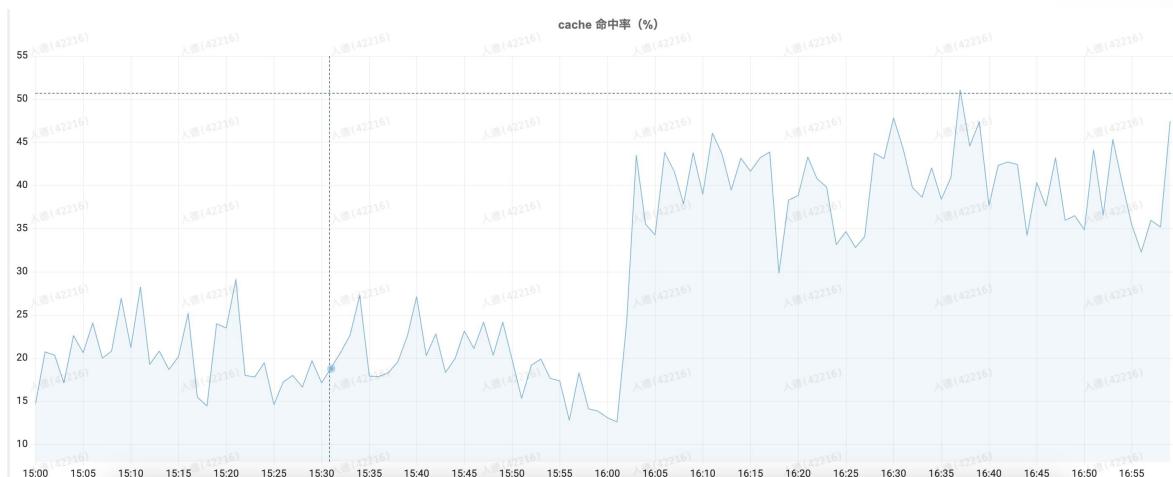
1. 过滤 (Lora、灰度 等匹配)
2. 每引擎节点计算得分
3. 排序 topK + 选择一个

$$\text{score} = W1 * \text{cache_ratio} - W2 * \text{request_load} - W3 * \text{prefill_load}$$

- cache_ratio: prefix cache 命中率
- request_load: 请求队列数
- prefill_load: 处于 prefill 阶段的 prompt 长度



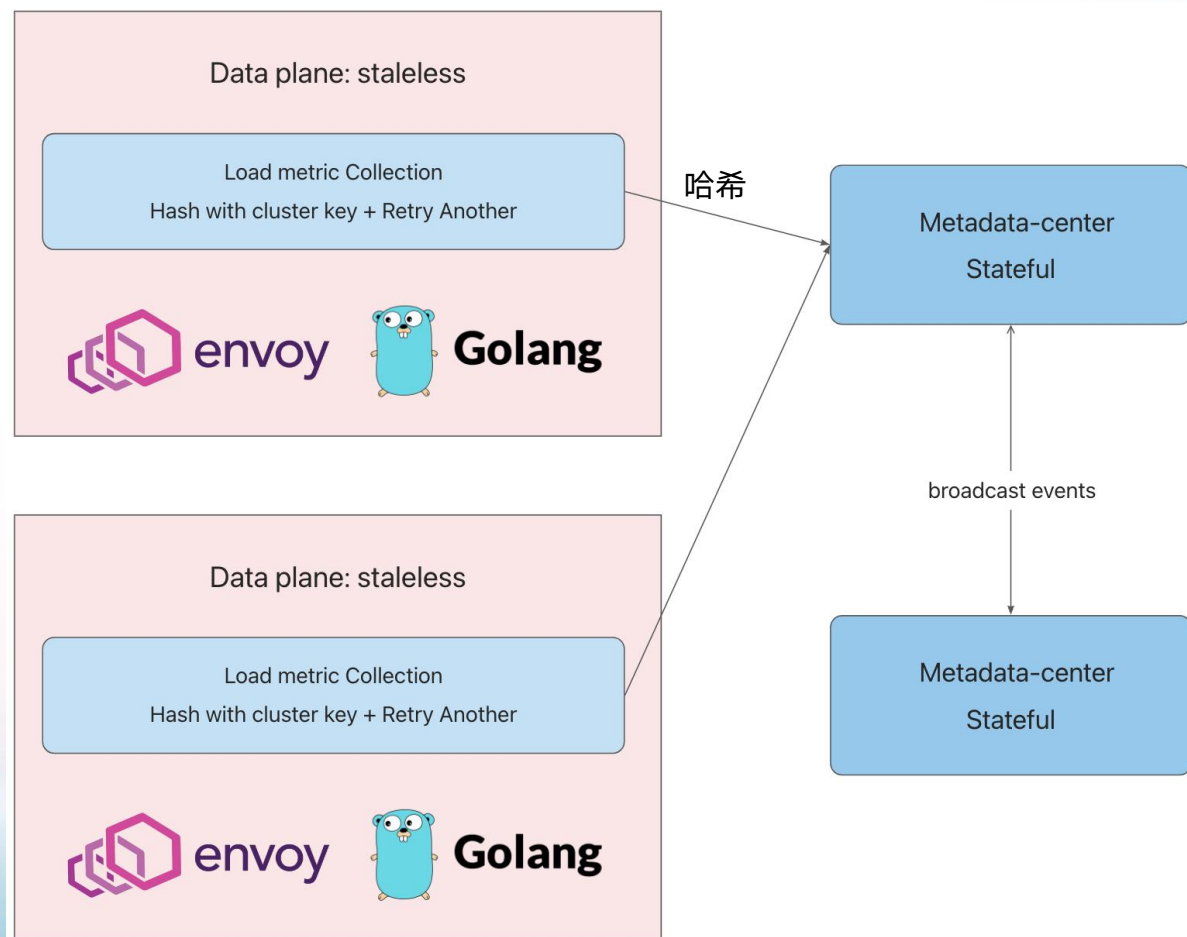
线上优化效果



- KVCache 命中率提升一倍
- TTFT 平均值降低 50%
- TTFT 长尾：数量级降低



亮点四：高可用架构设计



- 非强一致设计：最终一致性
 - 哈希：同一模型集群固定一个主节点
 - 异步广播事件同步
- 高可用设计
- 高性能
- 水平扩展

CONTENT

目录

01 推理场景对网关的挑战

02 AIGW 技术方案选型

03 AIGW 未来发展规划

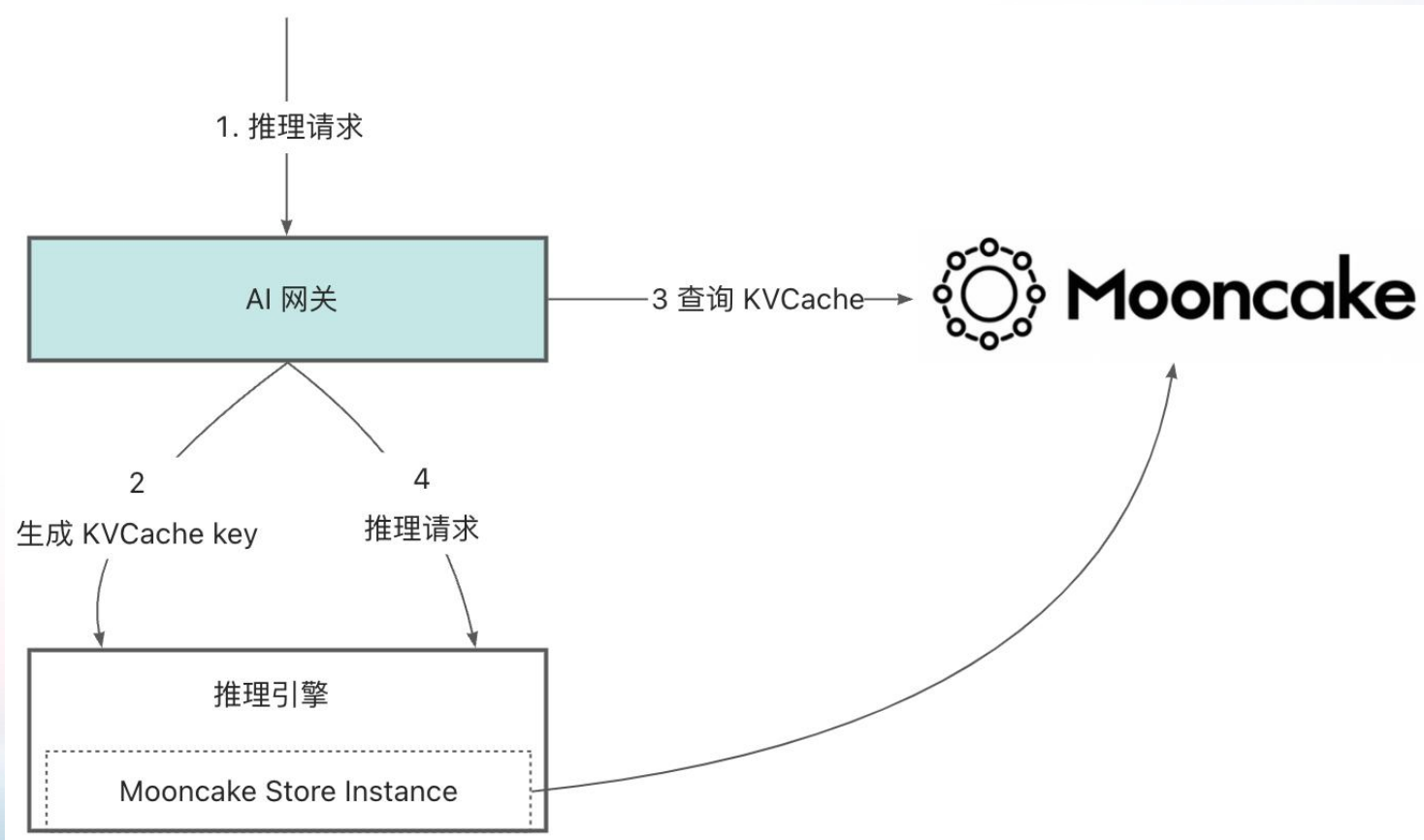
未来挑战

- 架构演进
 - 精确 cache-aware
 - 分离式推理
- 业务演进
 - 多模态、全模态
 - 长上下文、DSA、KDA
 - 超节点（大 DP）
- 规模化挑战
 - 多租共享 QoS
 - SLO-aware：基于时延预测

- 开源
 - Co-Design：社区协作
 - 标准化

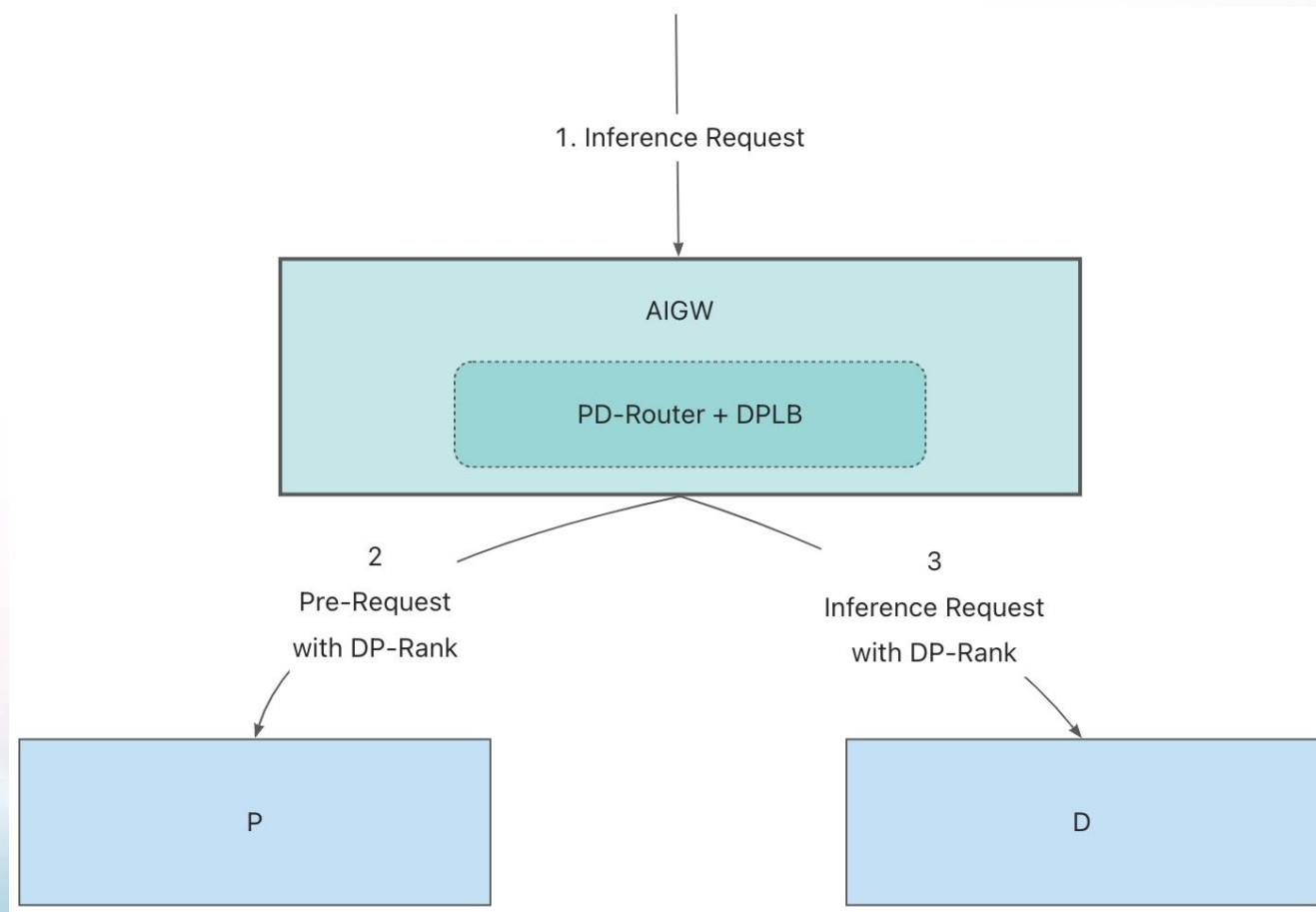


架构演进 — 精确 cache aware



- 近似 Cache-aware: 准确度 80%
- 提升 KVCache 本地命中率:
 - $L1 > L2 > L3$
- 减少传输时延

架构演进：PD 分离+ DPLB



PD 分离可以简化算法：

- P 节点: `cache ratio + prefill_load`
- D 节点: `request_load + token_load`

大 DP 场景下，DP 均衡性影响很大

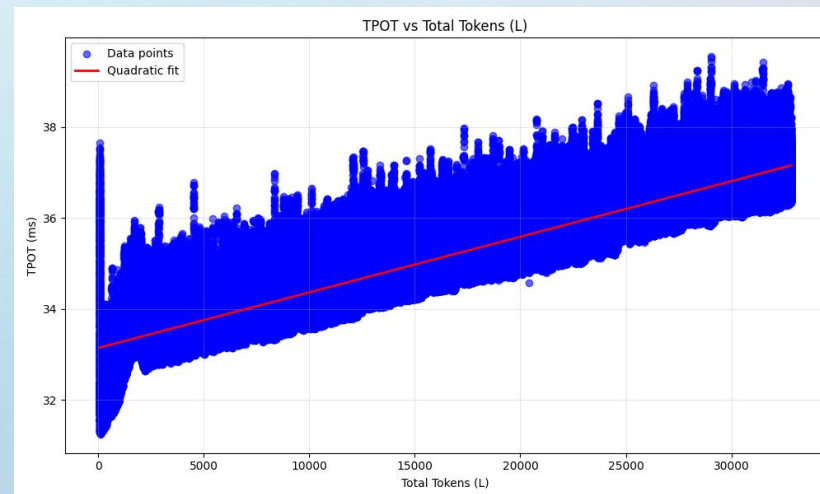
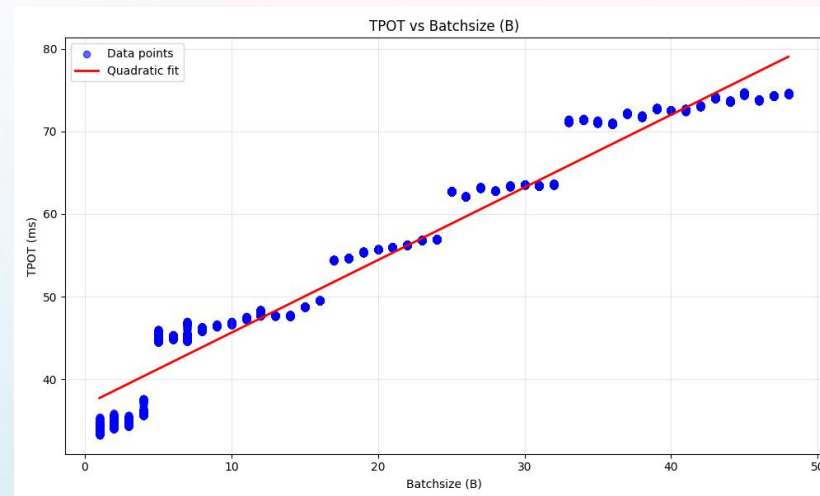
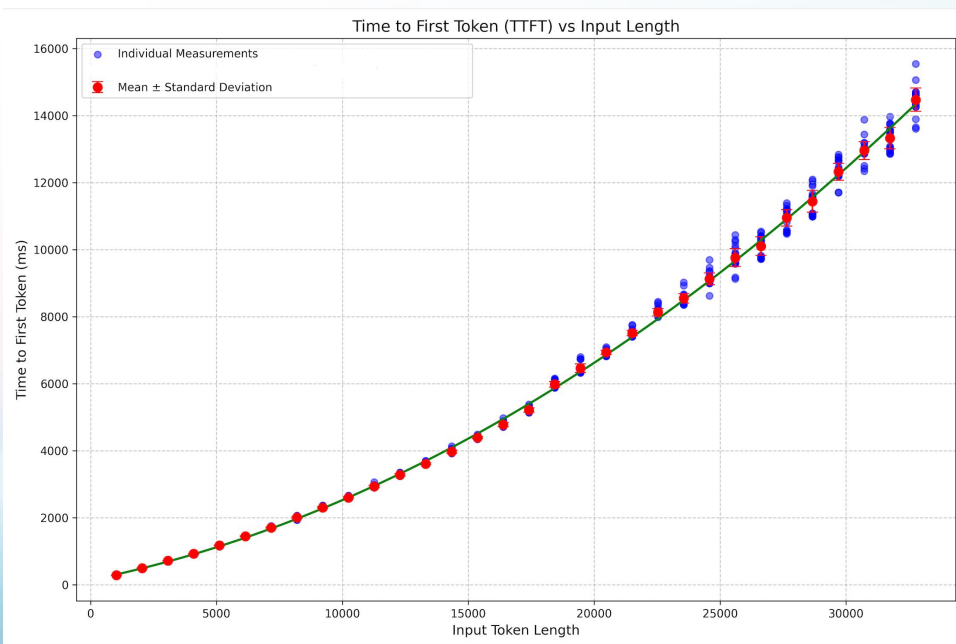
- 原因：DP 之间同步操作
- 目标：DP 之间计算均衡（Token 量）

规模化挑战：多租共享 QoS

场景：混合资源池，不同优先级请求

目标：满足 SLO 时，实现优先级调度

方案：SLO-aware，每请求时延预测



开源

群聊：AIGW 技术交流群



该二维码7天内(11月17日前)有效，重新进入将更新

<https://github.com/aigw-project/aigw>

- 欢迎测试体验：基于标准 Istio/Envoy 基础镜像
- 欢迎协作、共建